

```
// Runtime Polymorphism
// Dynamic Binding
// Late Binding
// Dynamic method dispatch possible for overridden methods. But this logic not applicable
to variables.
// To use Dynamic method dispatch, child class method must need to be in the parent class.
// Object Casting possible - Up Casting (Implicit) and Down Casting (Explicit)
// Use of "instanceof" operator - to check if object is of selected class or not
//--> Syntax: objectname instanceof ClassName Ex: a instanceof A
// instanceof returns boolean - true or false

public class DynamicMethodDispatch {
    public static void main(String[] args) {
        A a = new A(); // Valid
        A a1 = new B(); // Valid Implicit UpCasting
        B b = new B(); // Valid
        // B b1 = a; // Invalid // C.E. : Type mismatch: cannot convert from A to B
        // B b1 = (B) a; // C.E. : java.lang.ClassCastException: class A cannot be cast to
class B
        // B b2 = a1; // Type mismatch: cannot convert from A to B
        B b2 = (B) a1; // Valid Explicit DownCasting
        a.m1();
        a1.m1();
        a1.m2();
        b.m2();
        b2.m1();
        b2.m2();
        System.out.println(b instanceof A); // true
        System.out.println(a instanceof B); // false
        System.out.println(a instanceof A); // true
        System.out.println(a1 instanceof A); // true
        System.out.println(b2 instanceof A); // true
    }
}

class A {
    void m1() {
        System.out.println("Class A m1 Method");
    }

    void m2() {
        System.out.println("Class A m2 method");
    }
}

class B extends A {
    void m2() {
        System.out.println("Class B m2 Method");
    }
}
```

```

/*
Abstraction:
--> Hiding internal Implementation
--> Keyword: abstract (Can be used with class and/or methods)
--> abstract class cannot be instantiated
--> to use abstract class, it needs to be extended.
--> After extending, use child class object and parent class reference or child class
object and reference to use abstract class data

Abstract Methods:

--> Only declare abstract method in abstract class, never perform method implementation
--> Abstract methods can not have body
--> Class must be abstract if class have at least one abstract method.
--> Abstract class can have any method abstract or non-abstract.
--> final and abstract combination never possible (Illegal combination)
--> abstract method can not be declared as static
--> All abstract methods must be implemented in child class.
--> If child class implements all abstract methods, then parent class can be
implemented.
--> Abstract class can have constructor.
--> Static variable of abstract class can be called using abstract classname or Child
object.
--> Main method can be considered in abstract class also.

*/

abstract public class Abstract1 {
    void m1() {
        System.out.println("Regular m1 method");
    }

    abstract void m11();

    abstract void m111();
}

class ImpAbstract1 extends Abstract1 {
    // All abstract methods must need to implement in sub class
    void m11() {
        System.out.println("Abstract Implemented m11 method");
    }

    public void m111() {
        System.out.println("Abstract Implemented m111 method");
    }
}

class Main {
    public static void main(String[] args) {
        Abstract1 obj1 = new ImpAbstract1(); // Valid
        // ImpAbstract1 obj2 = new ImpAbstract1(); // Valid
        obj1.m1();
        obj1.m11();
        obj1.m111();
    }
}

```

```
abstract public class Abstract2 {
    void m2() {
        System.out.println("Regular m2 method");
    }

    abstract void m22();

    abstract void m222();
}

abstract class ImpAbstract2 extends Abstract2 {
    public void m22() {
        System.out.println("Abstract Implemented m22 method");
    }
}

class ImpAbstract22 extends ImpAbstract2 {
    public void m222() {
        System.out.println("Abstract Implemented m222 method");
    }
}

class Main1 {
    public static void main(String[] args) {
        ImpAbstract22 obj1 = new ImpAbstract22();
        obj1.m2();
        obj1.m22();
        obj1.m222();
    }
}
```

```
// Abstract class can have constructor
public abstract class Abstract3 {
    int num;
    String fname;

    public Abstract3(int num, String fname) {
        this.num = num;
        this.fname = fname;
    }

    public Abstract3() {
        System.out.println("Default constructor Called");
    }

    void display() {
        System.out.println("Abstract 3 Display method Called");
    }
}

class Main2 extends Abstract3 {

    String lname;

    Main2(int num, String fname, String lname) {
        super(num, fname);
        this.lname = lname;
    }

    Main2() {
        System.out.println("Main Default Constructor Called");
    }

    void display() {
        System.out.println("Main 2 Display method Called");
    }

    public static void main(String[] args) {
        Abstract3 a = new Main2(1, "abc", "xyz");
        a.display();
    }
}
```

```
/*
```

```
Interface :
```

```
--> Interface is a java feature, it will allow only abstract methods.
--> For interfaces, we are able to create only reference variables, we are unable to create objects.
--> In the case of interfaces, by default, all the variables are
    " public static final "
--> In the case of interfaces, by default, all the methods are
    " public and abstract"

--> In java applications, constructors are possible in classes and abstract classes
    but constructors are not possible in interfaces.

--> Interfaces provide more sharability in java applications when compared with classes and abstract classes.

--> In java applications, if we declare any interface with abstract methods then it is
    a convention to declare an
    implementation class for the interface and it is a convention to provide implementation
    for all the
    abstract methods in implementation class.
```

```
*/
```

```
package Interface;
```

```
public interface Interface1 {
```

```
    void m1();
```

```
    void m2();
```

```
    void m3();
```

```
}
```

```
class A implements Interface1 {
```

```
    public void m1() {
        System.out.println("m1 - A");
    }
```

```
    public void m2() {
        System.out.println("m2 - A");
    }
```

```
    public void m3() {
        System.out.println("m3 - A");
    }
```

```
    public void m4() {
        System.out.println("m4 - A");
    }
```

```
}
```

```
class Main1 {
```

```
    public static void main(String[] args) {
        Interface1 i1 = new A();
```

```
i1.m1();  
i1.m2();  
i1.m3();  
// i1.m4(); // C.E: The method m4() is undefined for the type Interface1
```

```
A a = new A();
```

```
a.m1();
```

```
a.m2();
```

```
a.m3();
```

```
a.m4();
```

```
}
```

```
}
```

```

/*
    Use of interface and abstract class together

--> In Java applications, if we declare an interface with abstract methods then
    it is mandatory to provide implementation for all the abstract methods in the
    respective implementation class.
--> In this context if we provide implementation for some of the abstract methods at the
    respective implementation class
    then compiler will rise an error,
--> where to come out from the compilation error we have to declare the respective
    implementation class as an abstract class
--> and we have to provide implementation for the remaining abstract methods by taking a
    sub class for the abstract class.

*/

package Interface;

public interface Interface2 {
    void m1();

    void m2();

    void m3();
}

abstract class A1 implements Interface2 {
    public void m1() {
        System.out.println("m1 - A1");
    }

    void m4() {
        System.out.println("m4 - A1");
    }
}

class B1 extends A1 {
    public void m2() {
        System.out.println("m2 - B1");
    }

    public void m3() {
        System.out.println("m3 - B1");
    }
}

class Main2 {
    public static void main(String[] args) {
        Interface2 i2 = new B1();
        i2.m1();
        i2.m2();
        i2.m3();
        // i2.m4(); // C.E: The method m4() is undefined for the type Interface2

        B1 b1 = new B1();
        b1.m1();
        b1.m2();
        b1.m3();
    }
}

```

```
    b1.m4();  
}  
}
```

/*

*Multiple Inheritance using Interface**--> In Java applications, it is not possible to extend more than one class to a single class**but it is possible to extend more than one interface to a single interface.*

*/

package Interface;

```
public interface Interface3 {
    void m1();
}
```

```
interface Interface31 {
    void m2();
}
```

```
interface Interface32 extends Interface3, Interface31 {
    void m3();
}
```

```
class A3 implements Interface32 {

    public void m1() {
        System.out.println("m1 - A3 method");
    }

    public void m2() {
        System.out.println("m2 - A3 method");
    }

    public void m3() {
        System.out.println("m3 - A3 method");
    }
}
```

```
class Main3 {
    public static void main(String[] args) {

        Interface3 i3 = new A3();

        i3.m1();
        // i3.m2(); // C.E.
        // i3.m3(); // C.E.

        Interface31 i31 = new A3();

        // i31.m1(); // C.E.
        i31.m2();
        // i31.m3(); // C.E.

        Interface32 i32 = new A3();

        i32.m1();
        i32.m2();
    }
}
```

```
i32.m3();
```

```
}
```

```
}
```

/*

--> In Java applications, it is possible to implement more than one interface into a single implementation class.

*/

package Interface;**public interface** Interface4 {
 void m1();
}**interface** Interface41 {
 void m2();
}**interface** Interface42{
 void m3();
}**class** A4 **implements** Interface4, Interface41, Interface42 {**public void** m1() {
 System.out.println("m1 - A4");
}**public void** m2() {
 System.out.println("m2 - A4 method");
}**public void** m3() {
 System.out.println("m3 - A4 method");
}

}

class Main4 {
 public static void main(String[] args) {
 Interface4 i4 = **new** A4();
 i4.m1();
 // i4.m2(); // C.E.
 // i4.m3(); // C.E.
 Interface41 i41 = **new** A4();
 // i41.m1(); // C.E.
 i41.m2();
 // i41.m3(); // C.E.
 Interface42 i42 = **new** A4();
 // i42.m1(); // C.E.
 // i42.m2(); // C.E.
 i42.m3();

}

}

```
/*
```

```
--> In general, if we declare abstract methods in an interface then we have to implement
all that interface methods
    in more number of classes with variable implementation part.
--> In the above context, if we require any method implementation common to every
implementation class with fixed implementation
    then we have to implement that method in the interface as default method.
--> To declare default methods in interfaces we have to use "default" keyword in method
syntax like access modifier.
--> If we need to add new function in existing interface, then with the help of "default"
implementation in interface,
    we can provide this functionality to all child classes.
--> In interface, method can be private but we have to write body also in same class.
```

```
*/
```

```
package Interface;
```

```
public interface Interface5 {
    default void m1() {
        System.out.println("m1 - default");
    }

    // It is possible to provide more than one default methods within a single interface.

    default void m2() {
        System.out.println("m2 - default");
    }

    private void m3() {
        System.out.println("m3 - private");
    }
}

class A5 implements Interface5 {
}

class A51 implements Interface5 {
}

class Main5 {
    public static void main(String[] args) {
        Interface5 i5 = new A5();
        i5.m1();

        Interface5 i51 = new A51();
        i51.m2();
    }
}
```

```
/*
--> After Java8, it is possible to override default methods in the implementation classes.
*/

package Interface;

public interface Interface6 {
    default void m1() {
        System.out.println("m1 - default");
    }
}

class A6 implements Interface6 {
    public void m1() {
        System.out.println("m1 - A6");
    }
}

class A61 implements Interface6 {
}

class Main6 {
    public static void main(String[] args) {
        Interface6 i6 = new A6();
        i6.m1();

        Interface6 i61 = new A61();
        i61.m1();
    }
}
```

/*

Static Methods in Interfaces:

--> Up to JAVA7 version, static methods are not possible in interfaces
 --> but from JAVA8 version static methods are possible in interfaces in order to improve sharability.
 --> If we declare static methods in the interfaces then it is not required to declare any implementation class to access that static method,
 we can use directly interface name to access static method.
 --> Static method can not be overridden in the child class.

NOTE:

-> If we declare static methods in an interface then they will not be available to the respective implementation classes,
 -> we have to access static methods by using only interface names not even by using interface reference variable
 -> After JAVA8 version, interfaces will allow concrete methods along with either "static" keyword or "default" keyword.

*/

```
package Interface;
```

```
public interface Interface7 {
    static void m1() {
        System.out.println("m1 - static");
    }
}
```

```
class A7 implements Interface7 {
    void m1(){
        System.out.println("m1 - A7");
    }
}
```

```
class Main7 {
    public static void main(String[] args) {
        Interface7.m1();
        Interface7 i7 = new A7();
        // i7.m1(); // C.E.: This static method of interface Interface7 can only be
        accessed as Interface7.m1
        A7 a7 = new A7();
        a7.m1();
    }
}
```

```
/*
```

```
CASE 1:
```

```
--> If two interfaces contain a method with same signature and same return type  
then  
    in implementation class only one method implementation is enough.
```

```
*/
```

```
package Interface;
```

```
interface Left {  
    void m1();  
}
```

```
interface Right {  
    void m1();  
}
```

```
public class Interface8 implements Left, Right {  
    public void m1() {  
        System.out.println("m1-Interface8");  
    }  
}
```

```
/*
```

```
CASE 2:
```

```
--> if two interfaces contain a method with same name but different arguments then  
    in the implementation class we have to provide implementation for both methods  
    and these methods will act as overloaded methods
```

```
*/
```

```
package Interface;
```

```
interface Left1 {  
    void m1();  
}
```

```
interface Right1 {  
    void m1(int a);  
}
```

```
public class Interface9 implements Left1, Right1 {  
    public void m1() {  
        System.out.println("m1-Interface9");  
    }  
  
    public void m1(int a) {  
        System.out.println("m1-int Interface9");  
    }  
}
```

```
/*
CASE 3:
    --> If two interfaces contain a method with same signature but different return
types then
        it is not possible to implement both interfaces simultaneously.

*/

package Interface;

interface Left2 {
    void m1();
}

interface Right2 {
    int m1();
}

/*
 * public class Interface10 implements Left2, Right2 {
 *
 *
 * public int m1() {
 *
 *
 * }
 *
 *
 * }
 */
```